

The rule base is the program

formal guardrails for LLM agents, act II

Three services — a CMS, a ticket desk, a receivables tracker — written as rules, served by one domain-free engine, with a solver reviewing every change.

Research review with code: the developer experience, two sabotage episodes, one domain transfer — and act I (proofs beside the code) in a single slide. August 2026.

No formal-methods background assumed — the four concepts you need are on slide 3.

Agents now out-write our ability to check

- LLM agents produce code faster than humans can meaningfully review it. Human attention is the bottleneck — and it samples, it does not cover.
- Tests also sample: they check the runs you thought of. The bugs that matter live in the interactions you did not think of.
- If we want agents (and background jobs, and time) to act autonomously, we need a way to say what must never break — and have a machine enforce it.

The proposal

The developer writes down intent as rules and invariants. Agents write the rest. Reasoning engines sit between them and arbitrate: every change must provably keep the rules.

Code becomes cheap to regenerate. The spec becomes the durable asset.

Setting: a regular web SaaS application — CRUD/API handlers, authorization, tenancy, workflows, background jobs. Not kernels, crypto, or avionics: every technique here is judged against ordinary product teams, CI budgets, and mainstream languages at the edges.

All the formal methods you need today

Invariant

A sentence about the system that must be true at every moment. Example: “a user never sees another user's data.” You already write these — in comments, runbooks and post-mortems. Here they become machine-checkable.

Model

A small, faithful board-game version of your system: its states and legal moves. In act II the “model” is not beside the system — the rule base IS both the model and the running program.

Checker

A tireless adversary (here: the Z3 solver). It considers every situation the rules allow — thousands you would never test — hunting for one that breaks an invariant.

Counterexample

The checker's proof of failure: the exact situation that breaks the rule, with the rule that allowed it named. Not “something is wrong” but “an editor who authored this article can publish it, via `editor_decide`.”

Proofs beside the code: it works — and it has a tax

2 protocols falsified, then proven

P1: the model checker beat a careful engineer twice on an online DB migration (drain guard, IS-NULL backfill). Version 3 survived — and was later proven inductively, for any system size, and live (tracks I, J, M).

1 round to the full fix

P4: same bug, same model, same generic prompt — a stronger frozen gate turned a partial fix into the full fix. Invest in gate strength, not prompt steering.

Proofs at five levels

Inductive invariants, parameterized (machine-inferred by UPDR), liveness under fairness, noninterference via self-composition, and a Dafny proof of real session code (tracks I–M).

59 anomalies per run

P3: skip the guard against real Postgres under concurrent load and the model's predicted anomaly appears 59×/run; 0 with the proven choreography. The invariant is load-bearing, not decorative.

3.94× faster, never wrong

P5: an agent optimized the CMS freely inside the frozen gate; two correctness-breaking “optimizations” were absorbed mechanically. Counterexamples, not humans, did the reviewing.

The residual tax

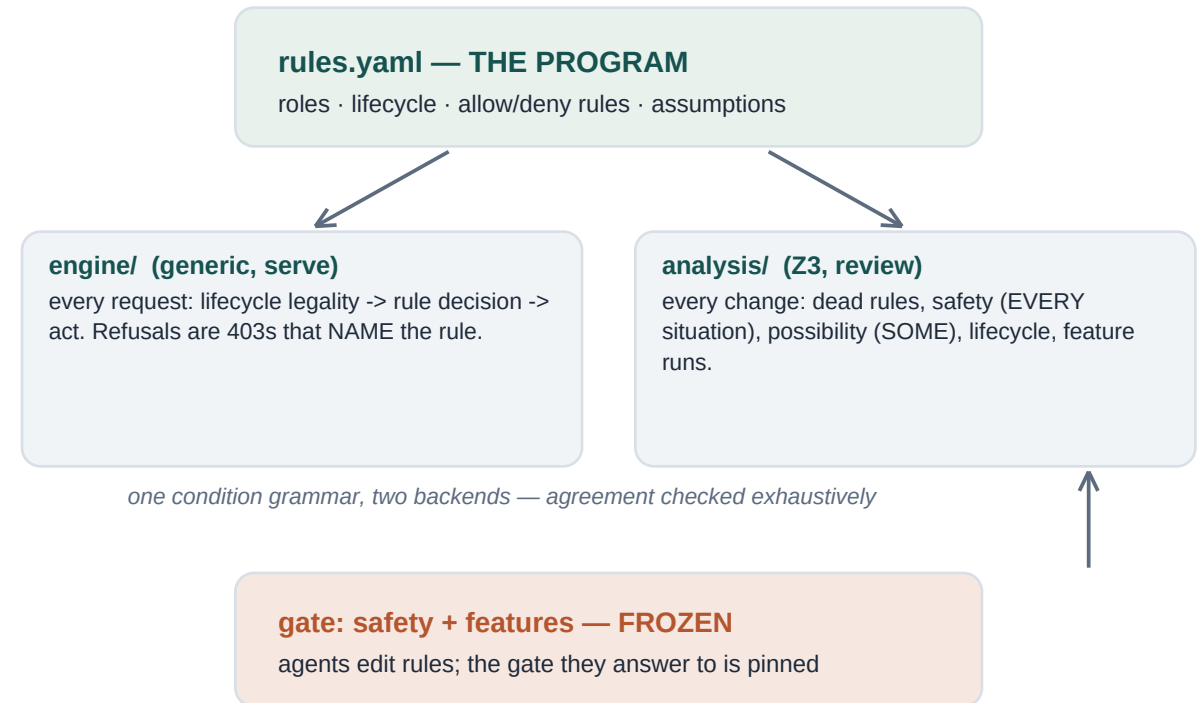
Every result above polices the model↔code boundary: MBT, trace validation, extraction fidelity. Act II removes that boundary for the ruled part of the system.

Language scouting closed the loop (notes 11–12): F* is not the SaaS rule language; of five ways proofs can meet code, “verified engine + small analyzable DSL” — Cedar's shape — ranked first. Act II builds exactly that.

“Is formal methods even the right framing?”

The developer's question, verbatim: all the examples model one facet of a system. If a human builds a full web service from the ground up — maybe formal methods is not the right framing. How about rule-based systems?

- Rule engines (OPS5, CLIPS, Drools, DMN) ran real businesses — and died of interaction opacity: at 10^3 rules nobody could ask “does any rule ever let X happen?”. That question is exactly an SMT workload.
- Formal methods stall on the opposite problem: the spec and the code are two artifacts, and they drift. Executable rules remove the seam — there is only one artifact for the domain.



The synthesis (note 13): rules are the programming surface · the solver is the reviewer · proof effort concentrates on the once-proven engine (Cedar's shape, note 12 pattern 1). Time, relations and computation still escalate up the assurance ladder.

A feature is a rule, not a handler

The whole CMS is one reviewable file: lifecycle plus fourteen allow/deny rules, each carrying the stakeholder sentence it translates. There are no handlers to write — endpoints exist because the lifecycle says so.

```
# rulesets/cms/rules.yaml — the CMS (130 lines)
lifecycle:
  transitions:
    - {action: submit, from: draft, to: in_review}
    - {action: publish, from: in_review, to: published}
    - {action: archive, from: published, to: archived}

  - id: no_self_decision
    description: "Nobody may publish or reject an
      article they authored themselves — separation
      of duties, with no exception for admins."
    effect: deny
    when: 'action in ["publish", "reject"]
      and actor.is_author'
```

```
$ curl -sX POST $CMS/articles/7/publish -H 'X-User: ed'
HTTP/1.1 403 Forbidden
{
  "error": "forbidden",
  "denied_by": "no_self_decision",
  "description": "Nobody may publish or reject an
    article they authored themselves...",
  "situation": {"role": "editor", "action": "publish",
    "is_author": true, "state": "in_review"}
}
```

- One vocabulary end to end: the ticket sentence, the rule id, the solver's finding, and the 403 body all say `no_self_decision`.
- Deny beats allow, silence means deny (Cedar/XACML semantics, stated once) — so admins are denied too: policy, not privilege, decides.

What must never happen — and what must keep working

The gate is two YAML files, frozen for agents. Safety quantifies over EVERY situation the rules allow; possibility demands witnesses (a “fix” that makes the system safely do nothing fails); features are frozen scenarios whose refusals are expected by name.

```
# safety.yaml — FROZEN for agents editing rules
safety:
  - id: S2_separation_of_duties
    requires: 'implies(action == "publish",
                      not actor.is_author)'
possibility:      # the anti-"does nothing" half
  - id: P2_review_by_non_author
    witness: 'action == "read" and not actor.is_author
              and resource.state == "in_review"'
lifecycle:
  only_into: {published: [publish]}

# features.yaml — refusals expected BY NAME
- {actor: ed,  action: publish,
   expect: deny,  denied_by: no_self_decision}
- {actor: erin, action: publish,
   expect: allow, state_after: published}
```

```
$ python -m analysis.analyze rulesets/cms
rules: 14 (5 deny, 9 allow) | situations: 8640
-- dead rules --
  ok: all 14 rules are effectual
-- assumptions --
  ok: creating roles are all assumable authors
-- safety: must hold in EVERY allowed situation --
  ok S1 ... S9  (9 properties)
-- possibility: must hold in SOME situation --
  ok P2_review_by_non_author
    witness: {role: editor, action: read, ...}
-- lifecycle --
  ok: 6 transitions live, gated entries respected
-- feature runs (frozen scenarios) --
  ok 5 features (34 steps)
VERDICT: PASS (0 findings)
```

- This runs in CI on every rule change, in seconds. Merging a rule diff means: no dead rules, every safety property proven over all situations, every workflow still alive, every scenario still passing.

The gate: frozen acceptance criteria, not the rules

The gate is what any version of the rule base must pass to count as correct. Agents edit rules.yaml freely; the gate they answer to is pinned somewhere they cannot reach.

```
# the same intent, twice – on purpose:
rules.yaml    - id: no_self_decision      effect: deny      # enforced: refuses requests, at runtime
safety.yaml   - id: S2_separation_of_duties requires: 'implies(action == "publish", not actor.is_author)'
                                                    # verified: EVERY situation, at review time
```

Deliberate redundancy

The rule is enforcement — the server refuses the request. The gate property is verification — the solver proves no allowed situation violates it. Delete the rule (the sabotage episode does) and the property catches it, with the concrete situation and the rule that now wrongly grants it.

Both directions, always

Safety alone is gameable: a rule base that denies everything passes every “never happens” property vacuously. So the gate pairs safety with possibility witnesses and frozen feature runs. strict_privacy broke NO safety property — it failed possibility P2 and two features.

Frozen mechanically, ratchets upward

analyze <edited-rules> --gate <pinned-dir>: the criteria live in a directory the editor cannot touch. Changing the gate is a human ceremony, and it only grows — each counterexample becomes a permanent item. Otherwise an optimizing agent weakens the check instead of fixing the rules (guardrail 1).

Verdicts, not opinions

PASS means mergeable — nothing else to review for the ruled part. FAIL means named findings — a dead rule, a situation plus the granting rule, the blocking deny, the failing feature step — routed back to whoever edited.

“Couldn't the gate hold ordinary code?” It can — at a price

The gate's strength is not free-standing — it depends on what the program is made of. Same gate sentence, two programs:

The program is rules (act II)

A $\forall$ -check is a proof.

S2 holds in all 8,640 situations, decided in ~2 s — the rule language is deliberately too weak to be undecidable.

No seam.

The analyzer and the server evaluate the SAME parsed rules — backend agreement checked exhaustively. The gate examines the program, not a model of it.

Bounded edit surface.

Agents edit only the rule base; the engine consults it on every request; the language cannot express a bypass or a side door. The analyzer interprets 100% of the editable artifact.

Findings name causes.

Dead rules, blocking denies, granting rules — the repair loop consumes rule interactions directly.

The program is free code (act I — P4/P5)

$\forall$ becomes sampling.

“No situation exists where...” is undecidable for code (Rice). You get property tests — P3: 735 requests, 0 errors — evidence, not proof; or per-change code proofs (track L, extraction-fidelity gap).

The seam returns.

Checked artifact $\neq$ executed artifact: MBT, trace validation, conformance harnesses — half of act I polices that boundary.

Unbounded artifact.

Code can be wrong in ways no gate item mentions — a new endpoint, a leaking cache, a bypassed check. Act I needed boundary lint and Grant<Op> compile-error tokens just to police this.

Findings name symptoms.

A failing test points at behavior; the cause is yours to find.

The quiet dependency: the gate is WRITTEN in the rule vocabulary — its properties quantify over situations that exist because the rule base declares them. Define a gate for free code and you end up specifying half a rule language anyway, just without executing it.

The honest flip side: where behavior is computation — migrations, fuzzy matching, performance — rules can't express it, and the repo does exactly this: free code under a gate, paying the difference in harnesses, lints and proofs. That is the escalation ladder.

Two plausible edits, seven named findings

Sabotage episode 1: two locally-reasonable edits, held to the frozen gate (--gate points at the pinned copy).

```
# edit 1: legal adds a privacy rule (LEG-77)          # edit 2: separation of duties removed
+ - id: strict_privacy
+   effect: deny
+   when: 'action == "read" and resource.state != "published" and not actor.is_author'
- - id: no_self_decision                             # "admins found it annoying"
```

```
$ python -m analysis.analyze rulesets/cms-buggy --gate rulesets/cms
FAIL DEAD allow rule 'editor_read_all': it never grants anything
  (overridden where it overlaps: deny_inactive, strict_privacy)
FAIL S2_separation_of_duties
  counterexample: {role: editor, action: publish, is_author: true, state: in_review, ...}
  allowed by: editor_decide
FAIL P2_review_by_non_author: IMPOSSIBLE — blocked by deny rule(s): strict_privacy
FAIL feat_publish_lifecycle: step 4 (ed read): expected allow, got deny (rule: strict_privacy)
VERDICT: FAIL (7 findings)
```

- Four different detectors fire at once: the dead-rule check catches the silent masking that rotted 1980s rule bases; $\forall$ -safety catches the removed deny with a concrete situation and the granting rule named; $\exists$ -possibility catches the workflow the new deny killed — the classic safety-only blind spot; and the frozen features catch both, by name.

A background job is just another actor

Ticket SYND-9: import published articles from publisher feeds, nightly. The importer gets no back door — it is an unprivileged HTTP client (role importer), so everything interesting about it is policy:

```
# the pipeline cannot skip provenance...
- id: import_needs_provenance
  effect: deny
  when: 'actor.role == "importer"
        and action == "create"
        and not resource.has_source'

# ...and a stolen importer credential is contained:
- id: importer_scope
  effect: deny
  when: 'actor.role == "importer"
        and action not in ["create", "read",
                           "submit"]'
```

```
$ analyze rulesets/cms-import-naive --gate rulesets/cms
# the obvious draft: syndicate straight to published
# ("the publisher already reviewed it")
FAIL role 'importer' can create items, but the
  assumptions say it can never be an author –
  stale assumption: analysis would silently
  skip every importer-authored situation
FAIL lifecycle: 'syndicate' (draft -> published)
  enters 'published'; gate allows only ['publish']
FAIL S8_imports_have_provenance (+S9, +feature)
VERDICT: FAIL (5 findings)
```

- Falsifier RB1 (tickets land as rule diffs): held — engine/ untouched; the change was 1 role, 1 field, 3 rules, 2 clients.
- The episode forced two new generic gate kinds: stale-assumption detection (the silent-unsoundness class) and gated lifecycle entries — review cannot be skirted by inventing a new transition into published.

Same engine, different world: money you are owed

The transferability test, from the ticket: “the user describes the amount they're owed, the approximate payer name or exact payment reference, and the due date; the system provides a dashboard and email notifications about overdue payments; bank transaction emails settle claims.”

```
projections:          # time enters the vocabulary
- {name: is_past_due, kind: date_passed,
  field: due_date}

# the ticket sentence, as a rule:
- id: claim_needs_identification
  effect: deny
  when: 'action == "create"
        and not (resource.has_payer_name
                  or resource.has_reference)'
```

```
# even the clock bot obeys the calendar:
- id: no_premature_overdue
  effect: deny
  when: 'action == "mark_overdue"
        and not resource.is_past_due'
```

290 lines of YAML — the domain

Money truth only from the bank feed (admins bounce off only_feed_settles), absolute tenant isolation, append-only ledger, calendar law: 13 rules, 10 safety + 7 possibility properties, all solver-checked.

~100 generic engine lines — the transfer cost

One missing concept: TIME. Declared date projections + an engine clock (--today, /__clock test seam), multi-source transitions (settle from awaiting AND overdue), a feature-file clock (advance_days).

Client-side, correctly — the projection boundary

Email parsing, the fuzzy matching itself (exact reference, else normalized payer name + exact amount), reminder dedup, cron. Cross-item and approximate: beyond any per-situation vocabulary.

- The transfer worked and its cost is measurable — and conceptual, not volumetric: the engine was missing time, not receivables.

The demo transcript, verbatim

```
$ python receivables_demo.py (engine clock: 2026-08-11)
-- users register what they are owed --
  ok rita's dashboard shows exactly her 3 claims (uma's is invisible)
-- day 1: the bank's transaction emails arrive --
  ok matched by approximate name (JOHN SMITH) and exact reference (INV-2026-001)
  ok the stranger's transfer is held for manual review, settles nothing
-- day 1: the overdue sweeper runs (it attempts EVERY awaiting claim) --
  ok every attempt refused by name: [(3, 'no_premature_overdue'), (4, ...)]
-- 30 days pass (clock -> 2026-09-10); the sweeper runs again --
  ok claim 3 (due 08-15) is overdue; uma's (due 09-30) is protected
    reminder -> rita@example.com: Payment overdue: EUR 500.00 (due 2026-08-15)
-- day 2: the late payment finally lands --
  ok ACME S.R.O. matches the overdue claim 3; settle succeeds
  ok even the admin cannot fake money truth: 403 only_feed_settles
VERDICT: PASS
```

- The sweeper is deliberately dumb — it tries everything, and the rules hold the clock accountable, not vice versa. A buggy (or compromised) bot is bounded by policy the solver already checked over every situation.
- Late payments land because possibility W2 (“settle from overdue exists”) is part of the frozen gate — liveness guarded the design before the demo existed.

Three services, one engine, every claim executable

3 : 1

services per engine — CMS, tickets, receivables on 1,097 lines of domain-free Python (grep-provable: zero domain words outside docstrings).

313 : 1,097

lines of YAML that ARE the CMS vs the shared generic engine+analyzer. Receivables: 290. Tickets: 100.

36,096

situations checked exhaustively — runtime evaluation and the Z3 compilation agree on every one (8,640 + 576 + 26,880).

12

named findings across two sabotaged rule bases (7 + 5); zero false passes; every finding carries a rule name or a concrete situation.

What the analyzer asks of every rule-base change (seconds, in CI):

- Is any rule dead — does removing it change no decision? (rule rot, reversed: two redundant guards were proven useless and deleted)
- Do the safety properties hold in EVERY allowed situation? If not: the situation + the granting rule.
- Is every workflow still POSSIBLE? If not: the blocking deny, found by re-checking without each deny.
- Are the assumptions consistent with the rules? (every creating role must be an assumable author)
- Is the lifecycle live and are gated states entered only through their gate (only_into)?
- Do the frozen feature scenarios still run — with every expected refusal denied by the right rule?

Kept deliberately sharp

The projection boundary

Rules see finite booleans (`is_author`, `has_source`, `is_past_due`). Fuzzy matching, cross-item invariants (“no two claims with one reference”, “ ≤ 3 published per author”) live in clients today — properly in store constraints (P9: the invariant $\rightarrow$ Postgres compiler that doesn't exist yet, anywhere).

No time-interleaving

Single-situation rules can't see races: the demoted author who is still `is_author`, the stale session. That is rung 2's job — the Quint temporal twin (falsifier RB4) stays on the ladder.

The clock is a trust root (RB6)

`is_past_due` is only as true as the engine's date. The analyzer proves “overdue only when past due” relative to the projection; a skewed clock breaks it silently. The temporal twin should model skew.

One entity per rule base

Receivables dodged it (transactions live with the bank; only claims are ruled). Linked ruled entities — `invoice` $\leftrightarrow$ `payment` $\leftrightarrow$ `dunning` — need N rule bases plus client joins, or engine work on relations.

The engine is unproven Python

Fine for research; the production form is a verified engine — Cedar's Lean proofs, or our own track-D Dafny $\rightarrow$ Go shape — so trust rests on one proof plus per-change analysis.

What we learned the hard way — now enforced

1 · The spec is frozen for agents

Enforced mechanically — the analyzer takes --gate from a pinned directory; agents edit rules, never the gate. Same contract as act I's frozen invariants.

3 · Gate strength beats NL steering

Same bug, model, prompt: the stronger gate turned a partial fix into the full fix (P4). Cheaper models are fine when the gate is strong.

5 · Translate / validate split

LLM translation of NL→rules is unsound — always followed by a sound solver step. Humans review rules sentence-by-sentence, never agent edits.

7 · Verify sub-agent claims

The checker corrected the author repeatedly (two “correct” protocols, two “needed” rules). Dead ends and falsified predictions are recorded in the notes.

2 · Gates need both directions

Safety-only gates accept fixes that trade away liveness (“the safest system does nothing”). Every gate here = safety + possibility witnesses + frozen feature runs.

4 · Escalate to proofs

Rules (this deck) < temporal models < inductive < parameterized < liveness < hyperproperties. Climb only when the ticket needs time, relations, or computation.

6 · Counterexamples are the currency

Machine-readable and named: unsat blockers, situation witnesses, denied_by 403s. One vocabulary from ticket to runtime error.

8 · Boundaries by construction

Bots are ordinary actors with no back door; capability tokens make bypass a compile error (track G). Typestate over discipline.

Falsifiers on the table

Held so far

- RB1 (tickets land as rule diffs, engine untouched): held twice — imports cost zero engine lines; the receivables transfer cost ~100 generic lines for one missing concept (time). Prediction: gate-language growth flattens as new domains reuse projections, only_into, the assumption check.
- Act I stands underneath: the migration protocol and CMS model remain proven at five levels (tracks I–M); the P4/P5 loop is the template for RB5.

Next

- RB3 — the LLM fidelity episode: same tickets, ticket→rule-diff vs ticket→handler-code; measure human corrections needed. The bet: declarative diffs review better.
- RB5 — agent repair: hand an agent the sabotaged rule bases under the frozen gate; expect ≤ 2 rounds (P4's result, cheaper, because diffs are rules).
- RB4 — the temporal twin in Quint: the demoted-author race and clock skew that per-situation rules provably cannot see.
- RB2 — scale: exhaustive backend agreement is 26,880 situations (~80 s) for receivables; per-condition projected enumeration or symbolic equivalence when it hurts. P8 — the same rules as Cedar policies (verified engine, symbolic analysis). P9 — the invariant→Postgres-constraint compiler nobody has built.

Parked (consciously, with reasons recorded)

Alloy (Z3 covers it) · Quint→Rust codegen (no tooling) · Kani until unbounded domains arrive · Verus (proxy-blocked; Dafny stands in).

The one-sentence takeaway

The developer writes the rules and the gate; the solver reviews every change; agents, bots, and time do their worst inside the fence.

For the ruled part of the system there is no code to review — only a rule diff and a verdict.

Try it: `examples/rule-driven-cms/check.sh` — tests + 5 analyses + 5 live demos, one command

Read it: `research/13-rule-based-cms.md` (the framing + both episodes) · `research/INDEX.md` (all notes)

Act I evidence: `research/09, 10, 12` · prototypes `p1–p7` · `examples/cms` (the spec-beside-code twin)